

Spring AI



The Spring AI project provides a Spring-friendly API and abstractions for developing AI applications.

Its goal is to apply Spring ecosystem design principles, such as portability and modular design, to the AI domain and promote using strongly-typed data structures and APIs as the building blocks of an application.



At its core, Spring AI addresses the fundamental challenge of AI integration: connecting your enterprise **Data** and **APIs** with the **AI Models**.

Getting Started

The [reference documentation](#) includes a [Getting Started](#) guide.

Spring Boot Version Compatibility:

- **Spring AI 2.x.x** ([main](#) branch) - Spring Boot 4 . x
- **Spring AI 1.1.x** ([1.1.x](#) branch) - Spring Boot 3 . 5 . x

Project Resources

- [Reference documentation](#)
- [Javadoc](#)
- [Upgrade notes](#)
- [Awesome Spring AI](#) - A curated list of awesome resources, tools, tutorials, and projects for building generative AI applications using Spring AI
- [Spring AI Examples](#) - Contains example projects that explain specific features in more detail.
- [Spring AI Community](#) - A community-driven organization for building Spring-based integrations with AI models, agents, vector databases, and more.

Contributing

We welcome contributions of all kinds! Please read our [contribution guidelines](#) before submitting a pull request or an issue.

Features

This is a high level feature overview.

- Support for all major [AI Model providers](#) such as Anthropic, OpenAI, Amazon Bedrock, Google, Ollama, Mistral AI, DeepSeek, and more. Supported model types include:

- [Chat](#)
- [Embedding](#)
- [Text to Image](#)
- [Audio Transcription](#)
- [Text to Speech](#)
- [Moderation](#)
- Portable API support across AI providers for both synchronous and streaming options. Access to [model-specific features](#) is also available.
- [Structured Outputs](#) - Mapping of AI Model output to POJOs.
- Support for all major [Vector Store providers](#) such as *Amazon Bedrock Knowledge Base, Amazon S3, Apache Cassandra, Azure Vector Search, Chroma, Couchbase, Elasticsearch, GemFire, MariaDB, Milvus, MongoDB Atlas, Neo4j, OpenSearch, Oracle, PostgreSQL/PGVector, Pinecone, Qdrant, Redis, Typesense, and Weaviate.*
- Portable API across Vector Store providers, including a novel SQL-like [metadata filter API](#).
- [Tool Calling](#) - Permits the model to request the execution of client-side tools and functions, thereby accessing necessary real-time information as required.
- [Observability](#) - Provides insights into AI-related operations.
- Document injection [ETL framework](#) for Data Engineering.
- [AI Model Evaluation](#) - Utilities to help evaluate generated content and protect against hallucinated response.
- [ChatClient API](#) - Fluent API for communicating with AI Chat Models, idiomatically similar to the WebClient and RestClient APIs.
- [Advisors API](#) - Encapsulates recurring Generative AI patterns, transforms data sent to and from Language Models (LLMs), and provides portability across various models and use cases.
- [MCP \(Model Context Protocol\)](#) - First-class support via Boot Starters and MCP Java Annotations for building AI applications that consume MCP servers or expose Spring-based services to the AI ecosystem, with STDIO, SSE, and Streamable-HTTP transport support.
- Support for [Chat Conversation Memory](#) with pluggable persistent backends (JDBC, Cassandra, MongoDB, Neo4j, Redis) and [Retrieval Augmented Generation \(RAG\)](#).
- Spring Boot Auto Configuration and Starters for all AI Models and Vector Stores - use [start.spring.io](#) to select the Model or Vector Store of choice.

Building from source

You don't need to build from source to use Spring AI. If you want to try out the latest and greatest, Spring AI can be built and published to your local Maven repository:

```
./mvnw clean install
```

This command builds all modules, runs unit tests, and publishes artifacts to your local Maven repository.

Please read our [contribution guidelines](#) for more details.